Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 9

Roll No : 496

Name : Dhruv Limbasiya

Enrollment : 24031701027

01) Write a function to calculate BMI given mass and height. (BMI = mass/h**2)

In [2]:
```
def BMI(mass , height):
    return mass/(height**2)

mass = float(input("Enter Mass in Kg :"))
height = float(input("Enter Height in Meter :"))

result = BMI(mass, height)
print("BMI :",result)
```

```
BMI : 22.6421929718633
```

02) Write a function that add first n numbers.

In [4]:
```
def sum_n(n):
    return n*(n+1) // 2

n = int(input("Enter Nth number :"))
result = sum_n(n)
print("Sum :",result)
```

```
Sum : 55
```

03) Write a function that returns 1 if the given number is Prime or 0 otherwise.

In [10]:
```
def is_prime(n):
    if n < 2:
        return 0
    for i in range(2, int(n//2) + 1):
        if n % i == 0:
            return 0
    return 1

n = int(input("Enter number: "))
print(is_prime(n))
```

```
1
```

04) Write a function that returns the list of Prime numbers between given two numbers.

In [ ]:
```
def primes_between(a, b):
    primes = []
    for n in range(a, b + 1):
        if n > 1:
            for i in range(2, int(n//2) + 1):
                if n % i == 0:
                    break
            else:
                primes.append(n)
```

```
    return primes

a = int(input("Enter start: "))
b = int(input("Enter end: "))

result = primes_between(a, b)
print(result)
```

05) Write a function that returns True if the given string is Palindrome or False otherwise.

In [14]:
```
def is_palindrome(s):
    return s == s[::-1]

s = input("Enter a string :")
if is_palindrome(s.lower()):
    print("String is palindrome")
else:
    print("string is not palindrome")
```

```
String is palindrome
```

06) Write a function that returns the sum of all the elements of the list.

In [15]:
```
def list_sum(lst):
    return sum(lst)

lst = list(map(int, input("Enter list elements: ").split()))
print("Sum of List :",list_sum(lst))
```

```
Sum of List : 10
```

07) Write a function to calculate the sum of the first element of each tuples inside the list.

In [16]:
```
def sum_first_elements(tuples):
    return sum(t[0] for t in tuples)

n = int(input("Enter number of tuples: "))
tuples = []
```

```
for _ in range(n):
    t = tuple(map(int, input("|Enter tuple: ").split()))
    tuples.append(t)

print(sum_first_elements(tuples))
```

```
10
```

08) Write a recursive function to find nth term of Fibonacci Series.

In [20]:
```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

n = int(input("Enter n: "))

print(fib(n))
```

```
75025
```

09) Write a function to get the name of the student based on the given rollno.

Example: Given dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'} find name of student whose rollno = 103

In [21]:
```
def get_student(roll, stu):
    if roll in stu:
        return stu[roll]
    else:
        return "Roll number not found"

n = int(input("Enter number of students: "))
students = {}

for _ in range(n):
    r = int(input("Roll: "))
    name = input("Name: ")
    students[r] = name
```

```
roll = int(input("Enter roll to search: "))
print(get_student(roll, students))
```

```
Jay
```

10) Write a function to get the sum of the scores ending with zero.

Example : scores = [200, 456, 300, 100, 234, 678]

Ans = 200 + 300 + 100 = 600

In [22]:
```
def sum_0(lists):
    newList = []
    for i in lists:
        if i%10==0:
            newList.append(i)
    return sum(newList)

lists = list(map(int, input("Enter list elements: ").split()))
print(sum_0(lists))
```

```
600
```

11) Write a function to invert a given Dictionary.

hint: keys to values & values to keys

Before : {'a': 10, 'b':20, 'c':30, 'd':40}

After : {10:'a', 20:'b', 30:'c', 40:'d'}

In [25]:
```
def invert_dict(dict1):
    inv_di = {}
    for k,v in dict1.items():
        inv_di[int(v)] = k
    return inv_di

keys = list(input("Enter keys for dict :").split())
```

```
values = list(input("Enter value for dict :").split())
dict1 = {k: int(v) for k, v in zip(keys, values)}
inv_di = invert_dict(dict1)
print(inv_di)
```

```
{10: 'a', 20: 'b', 30: 'c', 40: 'd'}
```

12) Write a function that returns the number of uppercase and lowercase letters in the given string.

example : Input : s1 = AbcDEfgh ,Ouptput : no_upper = 3, no_lower = 5

In [27]:
```
def count_case(s):
    uppercase=lowercase=0
    for i in s:
        if i.isupper():
            uppercase += 1
        else:
            lowercase+= 1
    return uppercase , lowercase

s = input("Enter a String :")
uppercase , lowercase = count_case(s)
print("uppercase total :",uppercase)
print("lowercase total :",lowercase)
```

```
uppercase total : 3
lowercase total : 5
```

13) Write a lambda function to get smallest number from the given two numbers.

In [32]:
```
a = int(input("Enter Value of A :"))
b = int(input("Enter Value of B :"))
smallest = lambda a, b: a if a < b else b
print(smallest(a,b))
```

```
4
```

14) For the given list of names of students, extract the names having more that 7 characters. Use filter().

In [38]:
```
lists = list(input("Enter list elements: ").split())
new_lists = list(filter(lambda lists : len(lists)>7 , lists))
print(new_lists)
```

```
['Limbasiya']
```

15) For the given list of names of students, convert the first letter of all the names into uppercase. use map().

In [54]:
```
lists = list(input("Enter list elements: ").split())
new_lists = list(map(str.title, lists))
print(new_lists)
```

```
['Dhruv', 'Limbasiya']
```

16) Write udfs to call the functions with following types of arguments:

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Legngth Positional(*args) & variable length Keyword Arguments (**kwargs)
5. Keyword-Only & Positional Only Arguments

In [55]:
```
def fun1(a,b,c):
    print(a,b,c)
fun1(1,2,3)
```

```
1 2 3
```

In [56]:
```
fun1(c=3 , a=1 , b=2)
```

```
1 2 3
```

In [57]:
```
def add(a , b=0 , c=0):
    print(a+b+c)
add(1)
```

```
1
```

In [58]:
```
add(1,2,3)
```

```
6
```

In [59]:
```
add(1,2)
```

```
3
```

In [60]:
```
add()
# error missing 1 required positiom argument
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[60], line 1
----> 1 add()

TypeError: add() missing 1 required positional argument: 'a'
```

In [61]:
```
def fun2(*args):
    print(args)
fun2()
```

```
()
```

In [62]:
```
fun2(1,2)
```

```
(1, 2)
```

In [63]:
```
fun2(1,2,c=3)
# error got an uneunexpected keyword argument 'c'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[63], line 1
----> 1 fun2(1,2,c=3)

TypeError: fun2() got an unexpected keyword argument 'c'
```

In [64]:
```
def fun3(**kwargs):
    print(kwargs)
fun3()
```

```
{}
```

In [65]:
```
fun3(a=1 , b=2 ,c=3)
```

```
{'a': 1, 'b': 2, 'c': 3}
```

In [66]:
```
def fun4(a,b,c,d,/):
    print(a,b,c,d)
fun4(1,2,3,4)
```

```
1 2 3 4
```

In [67]:
```
fun4(1,2,c=3,d=4)
#error got some positional-only arguments passed as keyword arguments: 'c, d'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[67], line 1
----> 1 fun4(1,2,c=3,d=4)

TypeError: fun4() got some positional-only arguments passed as keyword arguments: 'c, d'
```

In [68]:
```
def fun5(*,a,b,c,d):
    print(a,b,c,d)
fun5(a=1,b=2,c=3,d=4)
```

```
1 2 3 4
```

In [69]:
```
fun5(1,2,c=3 , d=4)
#error takes 0 positional arguments but 2 positional arguments (and 2 keyword-only arguments) were given
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[69], line 1
----> 1 fun5(1,2,c=3 , d=4)

TypeError: fun5() takes 0 positional arguments but 2 positional arguments (and 2 keyword-only arguments) were given
```

In [ ]: